End-to-End CI/CD Pipeline for a Calculator REST API

CSE 816 Software Production Engineering · Mini-Project Report

Student	Digvijaysinh Pawar
Roll number	IMT2023099
Course / Faculty	CSE 816 Software Production Engineering, Term I 2026-27 / Prof. B. Thangaraju
GitHub repository	https://github.com/ImNotFound7/calculator-devops
Docker Hub image	https://hub.docker.com/r/imnotfound/calculator-devops
Tech stack	Java 17, Spring Boot 3.5, Maven, JUnit 5, Git/GitHub, Jenkins, Docker, Docker Compose, Nginx

1. Introduction

The goal of this mini-project is to take a small Spring Boot calculator service and automate its whole delivery path. After a commit is pushed to the `main` branch on GitHub, Jenkins picks up the change and compiles the code, runs the unit and integration tests, builds a Docker image tagged with the build number, pushes it to Docker Hub, redeploys the stack with Docker Compose, checks that it is healthy and emails me the result. No step needs manual intervention.

The application is intentionally simple; most of the effort went into the pipeline, the container image and the deployment. As required, the scope stops at Compose. Ansible, Kubernetes and monitoring will be added in the Capstone.

2. System Architecture

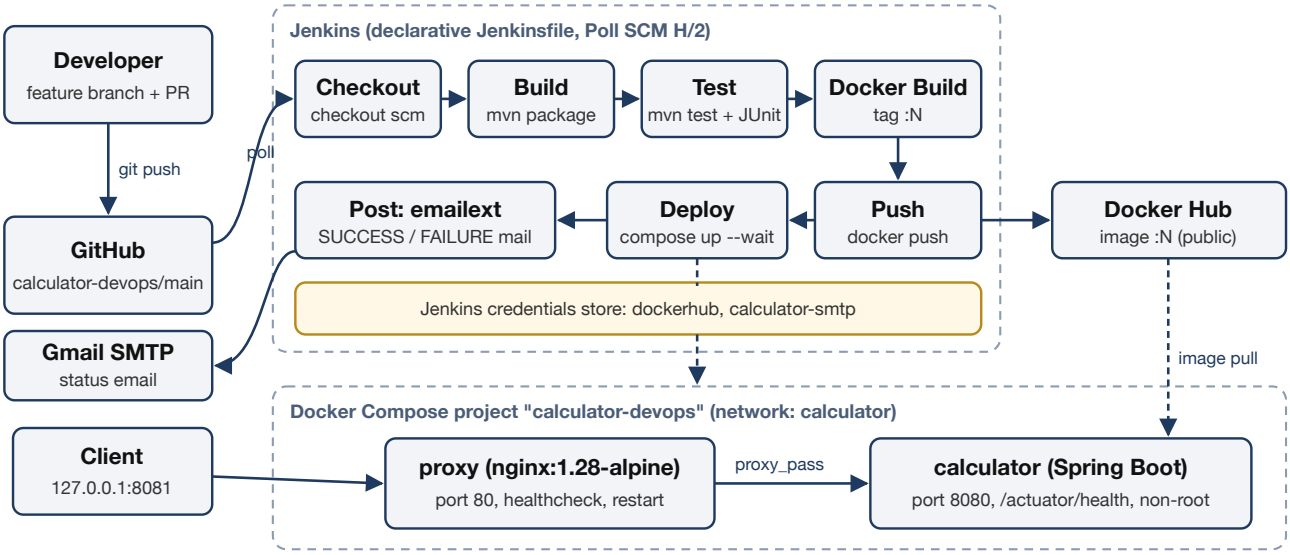


Figure 1: Pipeline and deployment architecture

Jenkins runs locally and deploys to the same Docker daemon. Compose runs two containers on a private bridge network. Nginx is the only container with a published port, so all client traffic reaches the API through the

proxy.

3. Application and Testing

The service is a Spring Boot (Spring Web + Actuator) application built with Maven into an executable JAR, `target/calculator.jar`. A single controller method maps `/api/v1/{operation}`, with the path restricted to the four supported operations, and delegates the arithmetic to `CalculatorService`.

Endpoint	Behaviour
GET /api/v1/add?a=&b=	Returns `{"operation":"add","a":2.0,"b":3.0,"result":5.0}`
GET /api/v1/subtract, /multiply	Same JSON shape with a−b and a×b
GET /api/v1/divide?a=&b=	Returns a/b; b = 0 (including −0.0) gives HTTP 400
GET /actuator/health	`{"status":"UP"}`, used by the Compose health checks

Validation. A `@RestControllerAdvice` returns HTTP 400 with an RFC 7807 problem-detail body (`type`, `title`, `status`, `detail`, `instance`) for missing, empty or non-numeric parameters, for `NaN` / `Infinity`, for division by zero and for results that overflow the `double` range. Unknown operations such as `/api/v1/power` return 404.

Tests. `mvn test` runs 31 JUnit 5 test cases, all passing:

- `CalculatorServiceTest` (14 cases): unit tests for every operation, division by +0.0 and −0.0, non-finite inputs, overflow and unknown operations.
- `CalculatorApiTest` (17 cases): `@SpringBootTest` + MockMvc integration tests that call the real endpoints, check the JSON fields of each operation, check the 400 error body for 11 invalid requests, check the health endpoint and the 404 for unknown routes.

Surefire writes JUnit XML reports to `target/surefire-reports/TEST-*.xml`, which Jenkins publishes.

4. Version Control (Git / GitHub)

- Public repository `ImNotFound7/calculator-devops` with a README that explains building, testing, running and calling the API.
- The application and pipeline were developed on the branch `feature/calculator-api` and merged into `main` through pull request #1. This report and the README cleanup were added through a second docs branch and pull request.
- Commits are small and use conventional prefixes (`feat:`, `ci:`, `docs:`), separating the application and tests from the pipeline and container work.
- `.gitignore` excludes `target/`, IDE files (`.idea/`, `.vscode/`, `*.iml`), OS files and local secrets (`.env`, `*.pem`, `*.key`). No credentials are committed; Docker Hub and SMTP secrets exist only in the Jenkins credentials store.

5. Jenkins CI/CD Pipeline

The pipeline is a declarative `Jenkinsfile` kept in the repository and loaded through *Pipeline script from SCM*. Its `triggers { pollSCM('H/2 * * * *') }` block makes Jenkins check GitHub about every two

minutes and start a build when `main` changes. Poll SCM was chosen over a webhook because Jenkins runs on a laptop (127.0.0.1) that GitHub cannot reach without a tunnel.

Stage	What it does
Checkout	`checkout scm` fetches the commit that triggered the build.
Build	`mvn -B -ntp clean package -DskipTests` compiles the code and produces the executable JAR.
Test	`mvn -B -ntp test` ; a `post { always { junit ... } }` block publishes the XML reports. A failing test fails the build, so no image is pushed or deployed.
Docker Build	`docker build -t imnotfound/calculator-devops:$BUILD_NUMBER .`
Push	`withCredentials` reads the `dockerhub` credential and logs in with `--password-stdin` using a temporary `DOCKER_CONFIG` directory, which is deleted afterwards so the token never stays on disk. The image is then pushed.
Deploy	`docker compose up -d --no-build --wait --wait-timeout 120` rolls out the new tag. A request to `/actuator/health` through the Nginx container then confirms the full path works, and `docker ps` is printed to the log.
Post	`emailext` emails the result (SUCCESS/FAILURE), build URL and image tag to my address via Gmail SMTP. On failure, the Compose status and logs are printed for debugging.

Credentials. Both secrets are *Username with password* entries in the Jenkins credentials store (`dockerhub` holds a Docker Hub access token, `calculator-smtp` holds a Gmail app password). Neither appears in the repository.

Re-runnability. Builds that touch the shared deployment run one at a time (`disableConcurrentBuilds()`). Compose only recreates containers whose image or configuration changed, and `--wait` marks the deploy as finished only after the health checks pass, so a re-run never breaks the running stack. Other options: `timestamps()` , a 20-minute `timeout` , and keeping the last 20 builds.

6. Containerisation (Dockerfile)

```
FROM maven:3.9.11-eclipse-temurin-17 AS build        # stage 1: build the JAR
WORKDIR /build
COPY pom.xml .
RUN mvn -B -ntp dependency:go-offline               # dependency layer is cached
COPY src ./src
RUN mvn -B -ntp package -DskipTests

FROM eclipse-temurin:17-jre-jammy                   # stage 2: JRE only, no JDK/Maven
RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/* \
    && groupadd --system calculator && useradd --system --gid calculator --home-dir /app calculator
WORKDIR /app
COPY --from=build --chown=calculator:calculator /build/target/calculator.jar app.jar
USER calculator                                     # non-root
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Best practices used: a multi-stage build (the runtime image contains only the JRE and the JAR); copying `pom.xml` before `src` so dependency downloads are cached between source changes; a dedicated non-root user; an explicit `EXPOSE` ; and a deny-by-default `.dockerignore` (`*` , then `!pom.xml` , `!src/`) that keeps

`target/`, `.git` and local files out of the build context. `curl` is the only extra package, installed for the health check. The image runs on its own with `docker run -p 8081:8080 imnotfound/calculator-devops:<tag>`.

7. Deployment (Docker Compose)

- **calculator**: the API image (`${CALCULATOR_IMAGE}`, which Jenkins sets to the new build tag). It sets the `SERVER_PORT` and `JAVA_TOOL_OPTIONS` environment variables, uses `restart: unless-stopped`, and its health check calls `/actuator/health` and expects `"status":"UP"`.
- **proxy**: `nginx:1.28-alpine` with a mounted `default.conf` reverse-proxying to `calculator:8080`. It starts only after the API is `service_healthy` and has its own health check through the proxy path. It is published on `127.0.0.1:8081`.
- Both services share a user-defined network, `calculator`; the API port is not exposed on the host.

A single `docker compose up -d` starts the full stack. Nginx resolves `calculator` through Docker's DNS (`resolver 127.0.0.11 valid=5s`), so it keeps working after Jenkins replaces the API container with a new image.

8. Results

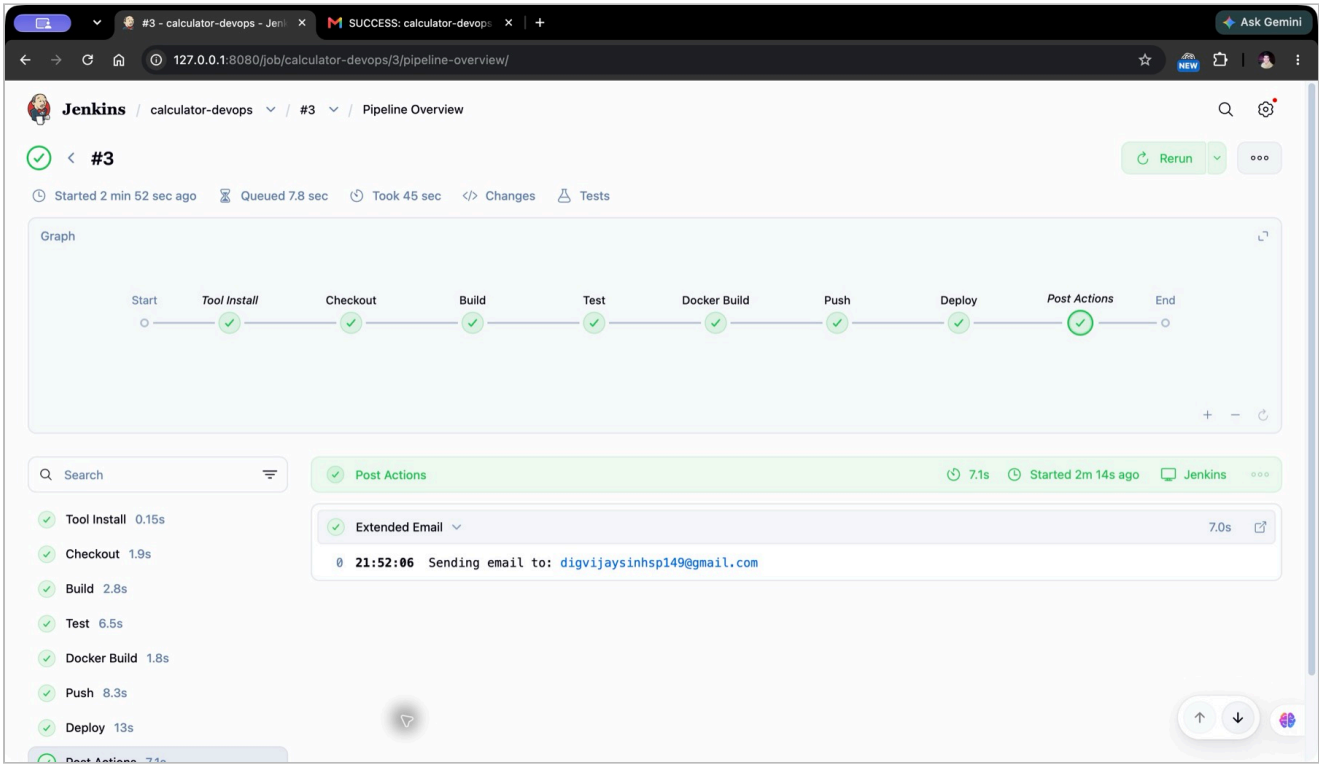


Figure 2: Green Jenkins run (build #3, started automatically by an SCM change): all stages passed and the email was sent

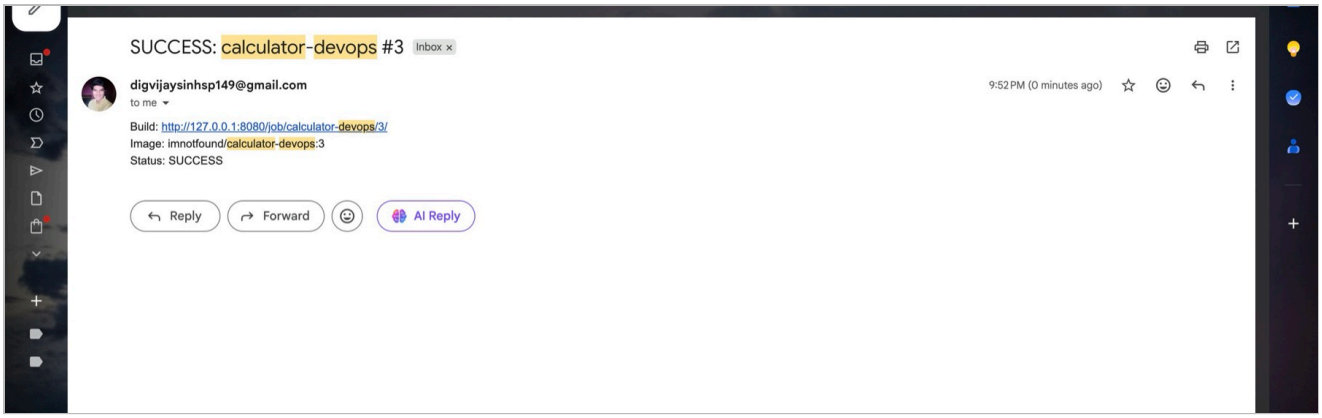


Figure 3: Build status email received in Gmail (SUCCESS, build #3, image tag :3)

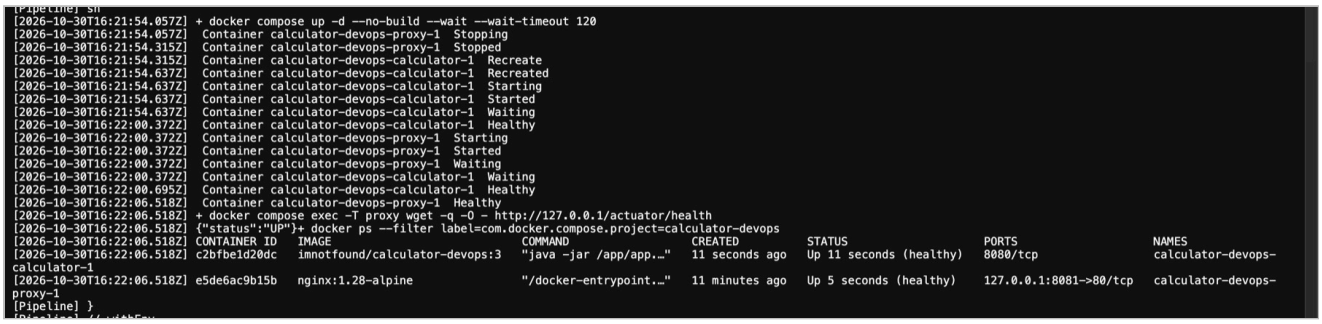


Figure 4: Deploy stage log: Compose recreates the API with image :3, both containers become healthy, then the health probe and docker ps run

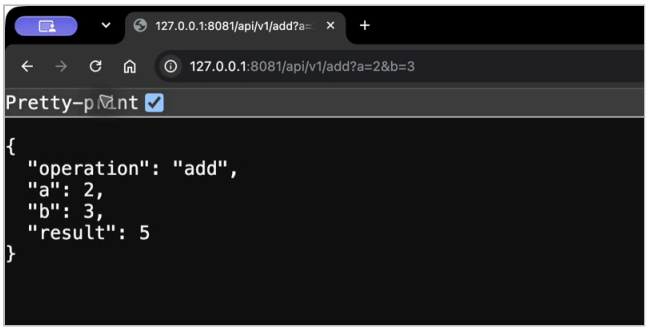


Figure 5: GET /api/v1/add?a=2&b=3

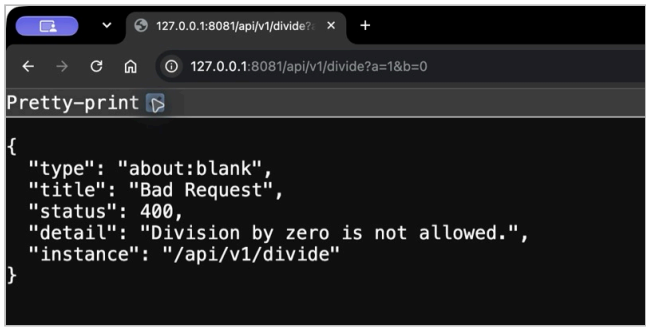


Figure 6: GET /api/v1/divide?a=1&b=0 returns 400

```
$ docker compose ps
NAME                            IMAGE                             STATUS                   PORTS
calculator-devops-calculator-1  imnotfound/calculator-devops:4  Up 7 minutes (healthy)  8080/tcp
calculator-devops-proxy-1       nginx:1.28-alpine               Up 7 minutes (healthy)  127.0.0.1:8081->80/tcp
$ curl -w ' [HTTP %{http_code}]' 'localhost:8081/api/v1/multiply?a=2.5&b=4'
{"operation":"multiply","a":2.5,"b":4.0,"result":10.0} [HTTP 200]
$ curl -w ' [HTTP %{http_code}]' 'localhost:8081/api/v1/add?a=abc&b=2'
{"type":"about:blank","title":"Bad Request","status":400,"detail":"Parameters 'a' and 'b' are
required and must be finite numbers.","instance":"/api/v1/add"} [HTTP 400]
$ curl 'localhost:8081/actuator/health'
{"status":"UP"}
```

Listing 1: Running stack (after Jenkins build #4) and sample responses through the Nginx proxy

9. Challenges Faced

- **Java version mismatch.** `java -version` showed Java 17, but Maven used `JAVA_HOME`, which pointed to Corretto 11, and failed with *release version 17 not supported*. I configured named `JDK17` and `Maven3`

tools in Jenkins and used them through the Jenkinsfile `tools` block, so builds no longer depend on the shell environment.

- **Port 8080 conflict.** Another project on my machine already published `0.0.0.0:8080`, so `localhost:8080` reached that service (HTTP 404) and not Jenkins, which listens on 127.0.0.1:8080. I used `127.0.0.1` for Jenkins and moved the calculator proxy to 8081 (configurable through `APP_PORT`).
- **Tools not found by the Jenkins service.** Jenkins runs as a macOS service and does not get the terminal's `PATH`, so `docker` was not found. The Jenkinsfile adds the Homebrew and Docker Desktop binary paths.
- **Proxy after redeploy.** Each deploy recreates the API container, which gets a new IP. A plain `proxy_pass http://calculator:8080` resolves the name only once, at Nginx startup, so the proxy would keep the old address. Putting the upstream in a variable with Docker's resolver makes Nginx re-resolve the name.
- **Gmail SMTP 535.** The first credential I saved in Jenkins was rejected with SMTP error 535. Gmail SMTP needs an app password, so I generated one, updated the `calculator-smtp` credential (TLS, port 587), and the emails were delivered after that.

10. Conclusion and Future Work

The project delivers a working pipeline: a push to `main` is built, tested, packaged as a versioned image on Docker Hub and deployed as a health-checked two-container stack, and the result is emailed. Build #3 ran this way, triggered only by a Git change. Next steps for the Capstone are to switch to a GitHub webhook (through a tunnel), add a `latest` tag and rollback to the previous tag, use Ansible to deploy to separate hosts, use Kubernetes for replicas and rolling updates, and add Prometheus/Grafana monitoring through the Actuator metrics endpoint.